

indic-nar-filler_asr : A-CMLM ASR head on a FROZEN data2vec-aqc large (Hindi, ALiBi)

```
raw audio          +-----+
(16 kHz mono)      | waveform x   (B, N_samples)   fp32, per-utterance LayerNorm (do_normalize) |
                    +-----+
                      |
                      v
#===== FROZEN data2vec-aqc large (313.3M, requires_grad=False, eval-pinned) =====#
# fairseq data2vec_audio body via Data2VecAQCEncoder -> CTC head + AQC (quantizer/project_q/EMA) #
# heads are DROPPED (not instantiated) #
# (1) Conv feature extractor 7 x Conv1d (512 ch) kernels [10,3,3,3,3,2,2] strides [5,2,2,2,2,2,2]#
# 320x downsample -> 50 fps extractor_mode=layer_norm (B,512,T)->(B,T,512) #
# 4.21M par ~5 GFLOP/s audio #
# (2) LayerNorm + feature projection Linear 512 -> 1024 (B, T, 1024) #
# (3) conv positional embed (k=128,g=16) + 24 x Transformer encoder layer (POST-norm) #
# each: MHSA(d=1024,H=16,dk=64) ->Add&LN-> FFN(1024->4096->1024,GELU) ->Add&LN #
# linear/token = 8d^2+4*d*ff = 25.2 MFLOP attn/token = 4*T*d 309M par ~33 GFLOP/s #
#=====|=====#
                      v      TAP = extract_features(layer=None) BEFORE CTC
                    +-----+
                    | tap (B, T, 1024) deterministic (encoder eval)|
                    +-----+
                      |
text_input_ids      (train only) | span-mask 20-30% of frames, spans len 10,
(B,T) Devanagari char ids + <mask> | replaced by learned mask_embed (1024)
                      |
                      v
+-----+          +-----+
| text_embed |      | + symmetric ALiBi position bias | (pos_mode=alibi: no PE add)
| 70 x 1024 |      +-----+
+-----+          +-----+
|------(ADD)-----> h = tap(+mask) + E_text(text_input_ids)
| fusion |
+-----+
                      |
                      v
#===== 8 x POST-norm Self-Attention layer (TRAINABLE) =====#
# MHSA(d=1024,H=16)+ALiBi ->Add&LN-> FFN(1024->4096->1024,GELU) ->Add&LN #
# linear/token = 25.2 MFLOP 100.6M par ~10 GFLOP/s audio #
#=====|=====#
                      v
                    +-----+
                    | Dropout(0.1) -> lm_head Linear 1024 -> 69 | logits (B, T, 69)
                    +-----+
                      |
                      v
framewise CrossEntropy on MASKED slots ONLY (CMLM), ignore_index=-100,
fill_weight down-weights <fill>.
target y = [<s>] + char_ids(| for space) + [</s>]x eos_repeat(3) + <fill> x (T-len)
n_keep = ceil(dur*50) = T -> Hindi ~12 char/s << 50 fps: 0% truncation, big <fill> tail
```


A. Why this shape at all (the A-CMLM idea, unchanged from filler_asr)

A-CMLM = Audio-Conditioned Masked LM. It is NON-autoregressive: instead of decoding left-to-right, it lays the whole character transcript on the acoustic frame axis (char i -> frame i), masks a random subset of those character slots, and predicts them from (a) the frozen acoustic tap and (b) the UNMASKED characters around them. At inference you start 100% masked and iterate mask-predict a few steps. The only thing that changed here is the acoustic encoder feeding the tap; the head, the masking, the targets and the loss are the filler_asr recipe.

B. FROZEN encoder (the ONE real change: HuBERT-xLarge -> data2vec-aqc)

The encoder is a frozen data2vec-aqc LARGE (SPRING-INX SSL checkpoint), loaded through fairseq via Data2VecAQCEncoder, which instantiates ONLY the inner data2vec audio body -- the CTC head and the AQC pretraining modules (quantizer, project_q, contr_proj, EMA) are physically absent, so the 'tap before CTC' is exactly extract_features(). It is a 320x conv stack (=> 50 fps, identical geometry to HuBERT) + 24 post-norm Transformer layers at d=1024. Shapes: wav (B,N) -> (B,T,1024), T = compute_output_length(N). It is pinned to eval() even under model.train(), so the tap is deterministic (no layerdrop/dropout) -- verified in smoke.py. Cost: ~313M params, ~33 GFLOP/s of audio (vs HuBERT-xLarge's 48 layers @ d=1280 ~ 102 GFLOP/s -- data2vec-aqc is ~1/3 the compute).

C. Tap -> span-mask -> position -> text fusion (all reused verbatim)

(1) tap (B,T,1024) is the frozen features. (2) TRAIN-ONLY span masking: 20-30% of the valid frames, in contiguous spans of 10 frames (~200 ms), are overwritten by a learned mask_embed (1024). This forces the SA stack to inpaint acoustic holes from context -- a robustness augmentation, gated on self.training. (3) POSITION: pos_mode=alibi injects a symmetric ALiBi bias into the attention (no sinusoidal add); ALiBi is what holds the </s>/tail structure on long clips. (4) TEXT: text_input_ids (Devanagari char ids, with <mask>=69 at masked slots) go through text_embed (70 x 1024) and are fused by ADDITION: h = tap(+mask) + E_text(text_ids). Small init means step 0 ~ audio-only, then the text pathway grows.

D. 8 self-attention layers -> lm_head -> loss (reused; only widths + vocab differ)

8 POST-norm Transformer layers at d=1024, H=16, ff=4096 (filler_asr used d=1280, ff=5120 to match HuBERT-xLarge). Then Dropout(0.1) -> lm_head Linear 1024 -> 69 -> per-frame logits (B,T,69). Loss = framewise CrossEntropy on the MASKED slots only (labels = target at masked positions, -100 elsewhere), with fill_weight down-weighting the dominant <fill> class. Head is ~100.8M trainable params, ~10 GFLOP/s audio. This is the entire trainable surface: SA + lm_head + text_embed + mask_embed; the encoder contributes 0 trainable tensors (asserted).

E. Targets, vocab and the 50-fps budget (Hindi-specific)

Targets built by the collator: normalize hi (NFC + drop ALL punctuation/symbol/foreign/zero-width) -> spaces become '|' -> per-CODEPOINT char ids. Devanagari is codepoint-level: each consonant, independent vowel, matra, virama ' ', nukta ' ', anusvara ' ', candrabindu ' ',

visarga 'ꣳ' is its OWN token, and conjuncts are spelled with the virama (ꣳꣳꣳ = ꣳ ꣳ ꣳ). Vocab = 63 Devanagari chars + '|' + <unk><pad><s></s> + <fill> + <mask> -> cfg.vocab size=69, <mask>=69 is input-only. Frame budget: data2vec gives 50 fps; Hindi char rate is p50=12, p99=19, max=29 tokens/s -> at 50 fps 0% truncation (median 283 spare frames). 50 fps is comfortable; the long <fill> tail is exactly why fill_weight exists.

CHANGES vs filler_asr A-CMLM (ref run: train_acmlm_alibi_fps50_fw003_exp150_4gpu)

aspect	filler_asr A-CMLM	indic-nar-filler_asr (this)
acoustic encoder	frozen HuBERT-xLarge ~963M, 48L d1280	frozen data2vec-aqc 313M, 24L d1024
encoder framework	HF transformers HubertModel	fairseq data2vec_audio (Data2VecAQCEncoder)
how CTC head removed	HubertModel (CTC never instantiated)	inner body only; CTC+AQc heads dropped
tap	last_hidden_state (B,T,1280)	extract_features (B,T,1024), before CTC
feature width d	1280	1024
input normalization	Wav2Vec2FeatureExtractor do_normalize	same (== fairseq layer_norm)
frame rate	50 fps (320x conv)	50 fps (same 320x conv)
SA head	8 layers, d1280, ff5120	8 layers, d1024, ff4096
lm head	1280 -> 33	1024 -> 69
text_embed	34 x 1280	70 x 1024
vocabulary	33 English char (a-z ' +fill)	69 (63 Devanagari + + specials + fill)
tokenization	Latin char	Devanagari codepoint (matra/virama/nukta split)
text normalization	lowercase + strip ASCII punct	NFC + drop ALL punct/symbol/foreign/zero-width
model integration	subclass HubertPreTrainedModel	encoder SWAP: self.hubert -> Data2VecEncoderAdapter
conda env	filler_asr (torch2.7, tf5.8, no fairseq)	indic-nar-filler_asr (clone of smear-moe-gemma3: torch2.4, tf5.5, fairseq 0.12.2)
training data	LibriSpeech 960h (English)	IndicASR / AI4B Hindi (~43k utts)
tap cache	442 GB precomputed memmap	live encoder forward (no cache yet)

UNCHANGED: A-CMLM masking (20-30% span-mask x10, force-full p=0.15), ADDITION text fusion, symmetric ALiBi, eos_repeat=3, positional <fill> targets, framewise-CE-on-masked loss + fill_weight, iterative mask-predict decode, 50-fps budget, ~100.8M trainable head.